FUNDAMENTOS DE SISTEMAS EMBEBIDOS

Práctica 3 — I2C y SPI: resolviendo la falta de ADC nativo

Materia	Fundamentos de Sistemas Embebidos	**Fecha**	Lunes 14 de septiembre de 2026
Horario	Virtual	**Lugar**	Laboratorio P003 - Virtual
Hardware	Raspberry Pi 4 + BMP280 + MCP3008 + potenciómetro	**Profesor**	Dr. Kevin Andrae Delgado Vargas

1. Objetivo general

Que el alumno se comunique con periféricos externos usando los protocolos I2C y SPI desde C, con las interfaces estables del kernel de Linux (i2c-dev, spidev) — y resuelva, por fin, la limitación mencionada desde la Práctica 2: la Raspberry Pi no tiene ADC nativo, así que para leer una señal analógica hace falta un chip conversor externo.

2. Objetivos específicos

- Habilitar I2C y SPI en la Raspberry Pi y verificar su funcionamiento.
- Escanear el bus I2C y detectar un dispositivo real conectado.
- Leer un registro de identificación (chip ID) de un sensor I2C desde C.
- Entender el protocolo de 3 bytes del ADC MCP3008 y leerlo por SPI desde C.
- Combinar I2C, SPI y GPIO (de la Práctica 2) en un solo programa integrador.
- Registrar los avances en el repositorio de Git del curso.

3. Antecedentes necesarios

Requiere el toolchain y repositorio de Git de la Práctica 1, y el manejo de GPIO por C de la Práctica 2. Se da por hecho el mapa de memoria y memory-mapped I/O — I2C y SPI son, en el fondo, dos formas distintas de estructurar la comunicación sobre esos mismos pines GPIO ya vistos en el pinout completo de la Práctica 2.

4. Material necesario

- Sensor de temperatura/presión BMP280 (módulo I2C)
- Convertidor analógico-digital MCP3008 (SPI, 8 canales, 10 bits)
- Potenciómetro de 10 kΩ
- Protoboard y cables jumper
- Laptop con acceso SSH, el toolchain de C, y el repositorio Git ya funcionando
- Raspberry Pi con el circuito de LEDs/botones de la Práctica 2

5. Alternativas de hardware

El BMP280 es económico y fácil de conseguir (entre $45 y $90), pero si alguien no lo tiene a tiempo hay dos rutas de salida. Lo importante en la Parte C es aprender el patrón de comunicación I2C: abrir el bus, fijar la dirección del esclavo, leer un registro. Medir temperatura es el vehículo, no el objetivo.

Opción	Qué cambia	Precio aprox.
BMP280	-	$45-90 MXN

DS3231	Módulo de reloj en tiempo real con sensor de temperatura integrado. Cambia la dirección I2C y el registro a leer; la lógica del código es prácticamente la misma.	$40-70 MXN
Cualquier módulo I2C disponible	Por ejemplo una pantalla LCD1602 con backpack I2C, común en casi cualquier kit. No da lectura de temperatura, pero permite practicar el mismo i2cdetect y la misma llamada a ioctl(I2C_SLAVE) — ajusten el objetivo de la Parte F según lo que tengan.	-

6. Diagrama de conexión

Componente	Pin GPIO (BCM)	Función
BMP280 — SDA	GPIO 2	Datos I2C (bus 1)
BMP280 — SCL	GPIO 3	Reloj I2C (bus 1)
MCP3008 — DOUT (MISO)	GPIO 9	Datos del ADC hacia la Pi
MCP3008 — DIN (MOSI)	GPIO 10	Datos de la Pi hacia el ADC
MCP3008 — CLK (SCLK)	GPIO 11	Reloj SPI
MCP3008 — CS	GPIO 8 (CE0)	Selección de chip

El potenciómetro se conecta con sus dos terminales externos a 3.3V y GND, y su terminal central (el cursor) al canal 0 del MCP3008 — actúa como un divisor de voltaje variable.

7. Desarrollo de la práctica

Parte A — Habilitar I2C y SPI

1. Conectarse por SSH a su Raspberry Pi.
2. La posición de estas opciones dentro del menú de raspi-config cambia entre versiones de Raspberry Pi OS (Bookworm, Trixie, etc.) — en vez de navegar el menú, usar el comando directo, que es el mismo sin importar la versión:

```
sudo raspi-config nonint do_i2c 0
sudo raspi-config nonint do_spi 0
```

3. Si prefieren el menú interactivo, la ruta es Interface Options > I2C / SPI — pero el número de sección puede no coincidir con lo que muestre algún tutorial en línea.
4. Alternativa manual, editando el archivo de arranque directo (misma ruta que ya usaron desde la Práctica 1):

```
sudo nano /boot/firmware/config.txt
# agregar al final:
dtparam=i2c_arm=on
dtparam=spi=on
```

5. Reiniciar y confirmar que los dispositivos existan:

```
sudo reboot
ls /dev/i2c-*
ls /dev/spidev*
```

6. Instalar las herramientas de línea de comandos para I2C:

```
sudo apt install -y i2c-tools libi2c-dev
```

Parte B — Escaneo del bus I2C

7. Conectar el BMP280 según el diagrama de la sección 5.
8. Escanear el bus para confirmar que el sensor responde:

```
i2cdetect -y 1
```

9. Deben ver una dirección resaltada en la cuadrícula — 0x76 o 0x77, dependiendo del módulo. Esa es la dirección que usarán en el código.

Parte C — Leer el chip ID del BMP280 desde C

10. Crear el archivo fuente:

```
nano bmp280_id.c
```

11. Escribir el siguiente programa — lee el registro 0xD0 ("id"), que para un BMP280 siempre debe regresar 0x58 (es el mismo patrón "WHO_AM_I" que existe en casi todo sensor I2C, para confirmar que se está hablando con el chip correcto):

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <linux/i2c-dev.h>

#define I2C_ADDR 0x76   // o 0x77, segun lo que muestre i2cdetect
#define REG_ID   0xD0

int main(void) {
    int fd = open("/dev/i2c-1", O_RDWR);
    if (fd < 0) { perror("open"); return 1; }

    if (ioctl(fd, I2C_SLAVE, I2C_ADDR) < 0) {
        perror("ioctl I2C_SLAVE");
        return 1;
    }

    // Escribir la direccion del registro que queremos leer
    unsigned char reg = REG_ID;
    if (write(fd, &reg, 1) != 1) { perror("write"); return 1; }

    // Leer el valor de ese registro
    unsigned char chip_id;
    if (read(fd, &chip_id, 1) != 1) { perror("read"); return 1; }

    printf("Chip ID leido: 0x%02X\n", chip_id);
    printf("%s\n", chip_id == 0x58 ? "Es un BMP280 real" : "Algo no cuadra");
```

```
    close(fd);
    return 0;
}
```

12. Compilar y ejecutar:

```
gcc bmp280_id.c -o bmp280_id
./bmp280_id
```

13. Si no ven 0x58, revisar la dirección I2C_ADDR contra lo que mostró i2cdetect en la Parte B.

Parte D — Entendiendo el protocolo del MCP3008

El MCP3008 no tiene registros como el BMP280 — cada lectura es una conversación de 3 bytes por SPI:

Byte	Qué se envía	Qué se recibe
1	0x01 (bit de inicio)	(sin importancia)
2	(8+canal) << 4 — selecciona el canal 0-7	Los 2 bits más altos del resultado
3	0x00 (relleno)	Los 8 bits bajos del resultado

El resultado final (10 bits, 0-1023) se arma combinando los bits de las respuestas 2 y 3. Conectar el MCP3008 y el potenciómetro según el diagrama de la sección 5 antes de continuar.

Parte E — Leer el potenciómetro por SPI desde C

14. Crear el archivo fuente:

```
nano leer_pot.c
```

15. Escribir el siguiente programa:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <linux/spi/spidev.h>

int main(void) {
    int fd = open("/dev/spidev0.0", O_RDWR);
    if (fd < 0) { perror("open"); return 1; }

    unsigned char mode = SPI_MODE_0;
    unsigned char bits = 8;
    unsigned int speed = 1000000;
    ioctl(fd, SPI_IOC_WR_MODE, &mode);
    ioctl(fd, SPI_IOC_WR_BITS_PER_WORD, &bits);
    ioctl(fd, SPI_IOC_WR_MAX_SPEED_HZ, &speed);

    while (1) {
        unsigned char canal = 0;
        unsigned char tx[3] = {1, (8 + canal) << 4, 0};
        unsigned char rx[3] = {0};

        struct spi_ioc_transfer tr = {
            .tx_buf = (unsigned long)tx,
            .rx_buf = (unsigned long)rx,
```

```
            .len = 3,
            .speed_hz = speed,
            .bits_per_word = bits,
        };
        ioctl(fd, SPI_IOC_MESSAGE(1), &tr);

        int valor = ((rx[1] & 3) << 8) | rx[2];   // 0-1023
        float voltaje = (valor * 3.3f) / 1023.0f;
        printf("Valor: %4d   Voltaje: %.2fV\n", valor, voltaje);

        usleep(300000);
    }
    close(fd);
    return 0;
}
```

16. Compilar y ejecutar, girando el potenciómetro mientras corre:

```
gcc leer_pot.c -o leer_pot
./leer_pot
```

17. El valor debe moverse entre 0 y 1023 conforme giran la perilla — 0 y 1023 son los extremos (0V y 3.3V).

Parte F — Reto final: sensor combinado, sin código de partida

Escriban un programa nuevo, sensor_combinado.c, que:

- Lea la temperatura ambiental leyendo el BMP280 por I2C cada 2 segundos (usar el registro de temperatura, no solo el chip ID — investigar el registro correspondiente con el datasheet o probando).
- Lea continuamente la posición del potenciómetro por SPI, y la use como un umbral configurable (mapear el valor 0-1023 a un rango de 0-50°C).
- Encienda un LED (de su circuito de la Práctica 2, por GPIO con libgpiod) cuando la temperatura leída supere el umbral que marca el potenciómetro.

Parte G — Commit con Git

18. Agregar los archivos nuevos al repositorio ya iniciado en prácticas anteriores:

```
git add bmp280_id.c leer_pot.c sensor_combinado.c
git commit -m "Practica 3: I2C (BMP280) y SPI (MCP3008)"
git log --oneline
```

8. Checklist de salida

- I2C y SPI habilitados, con /dev/i2c-1 y /dev/spidev0.0 presentes.
- i2cdetect muestra la dirección del BMP280 (0x76 o 0x77).
- bmp280_id.c compila y reporta chip ID 0x58.
- leer_pot.c funciona y el valor cambia al girar el potenciómetro.
- sensor_combinado.c funciona: enciende el LED cuando la temperatura supera el umbral del potenciómetro.
- git log --oneline muestra el commit de hoy.

9. Solución de problemas comunes

Problema	Causa probable / solución

i2cdetect no muestra ninguna dirección	Revisar cableado SDA/SCL; confirmar que I2C esté habilitado (ls /dev/i2c-*).
La opción de I2C/SPI no aparece donde dice el tutorial	El menú de raspi-config cambia de posición entre versiones de Raspberry Pi OS; usar sudo raspi-config nonint do_i2c 0 / do_spi 0 en vez de buscarla.
Chip ID no es 0x58	Verificar la dirección I2C_ADDR en el código contra lo que mostró i2cdetect — puede ser 0x76 o 0x77 según el módulo.
undefined reference al compilar SPI	Revisar que los includes sean exactos (linux/spi/spidev.h); no se necesita -l adicional para spidev, a diferencia de libgpiod.
El valor del potenciómetro no cambia	Confirmar que el cursor central del potenciómetro va al canal 0 del MCP3008, no a un extremo.
Permission denied en /dev/spidev0.0	sudo usermod -aG spi $USER (cerrar sesión y volver a entrar).

10. Puente a la Práctica 4

La siguiente práctica (P04 — Boot) explora el proceso de arranque de la Raspberry Pi — el paso previo a que cualquiera de estos periféricos esté disponible para su código.